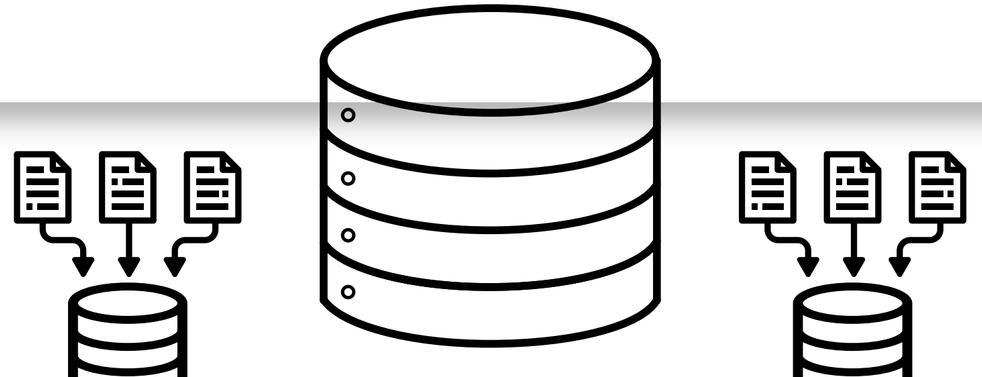
E-COMMERCE DATABASE MANAGEMENT SYSTEM

Understanding Users, Designing Better Experiences

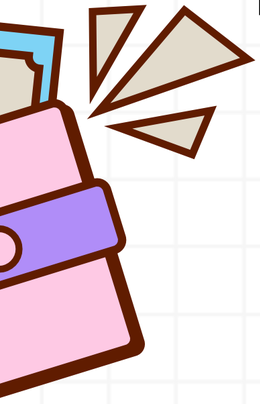
Presented by : Hari



Project Overview



The E-Commerce Database Management System is a MySQL-based project designed to manage customer details, product categories, products, orders, and payment transactions. The system stores data in related tables and uses SQL queries to retrieve, update, and analyze information efficiently.



Key Features:

- Customer management
- Product management
- Order processing
- Payment tracking
- Sales reporting



Problem Statement

Managing e-commerce information manually can lead to data duplication, errors, and difficulties in tracking orders and payments. The objective of this project is to develop a structured database that stores and manages customer information, products, orders, and payments efficiently. The system should support product management, order processing, payment tracking, sales reports, identification of best-selling products, and customer purchase history.







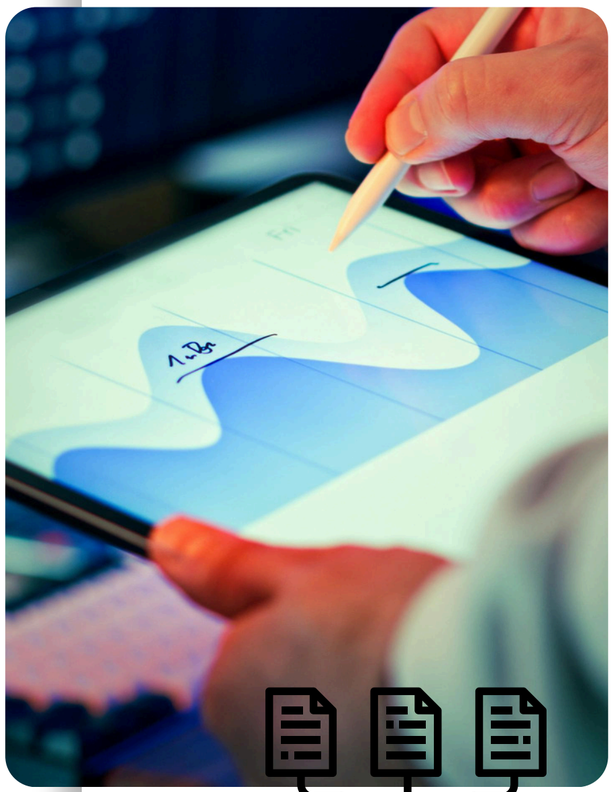






Effects

Objectives





- To store and manage customer information.
- To organize products under different categories.
- To maintain product prices and available stock.
- To record customer orders and order status

DATABASE DESIGN

E-COMMERCE DATABASE MANAGEMENT SYSTEM

Well-structured tables | Strong relationships
Better data integrity | Efficient operations

ER DIAGRAM

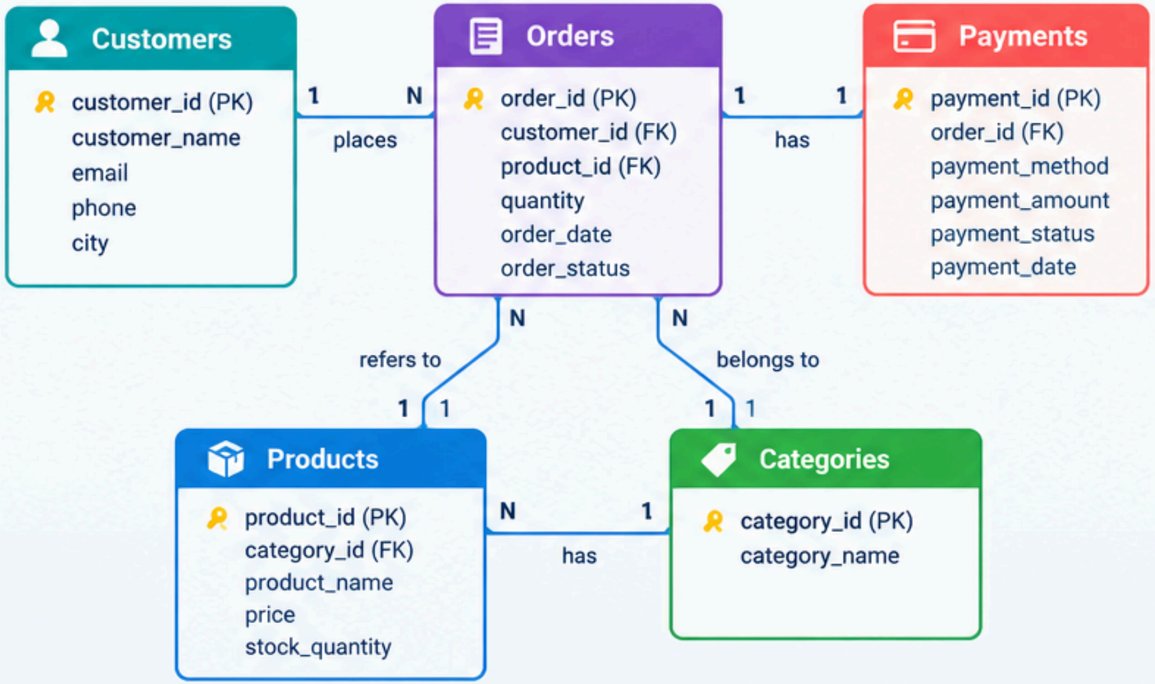


TABLE STRUCTURE

Customers

Field Name	Data Type	Key
customer_id	INT	PK
customer_name	VARCHAR(100)	-
email	VARCHAR(100)	-
phone	VARCHAR(15)	-
city	VARCHAR(50)	-

Orders

Field Name	Data Type	Key
order_id	INT	PK
customer_id	INT	FK
product_id	INT	FK
quantity	INT	-
order_date	DATETIME	-
order_status	VARCHAR(20)	-

Categories

Field Name	Data Type	Key
category_id	INT	PK
category_name	VARCHAR(60)	-

Products

Field Name	Data Type	Key
product_id	INT	PK
category_id	INT	FK
product_name	VARCHAR(100)	-
price	DECIMAL(10,2)	-
stock_quantity	INT	-

Payments

Field Name	Data Type	Key
payment_id	INT	PK
order_id	INT	FK
payment_method	VARCHAR(50)	-
payment_amount	DECIMAL(10,2)	-
payment_status	VARCHAR(20)	-
payment_date	DATETIME	-

RELATIONSHIPS

Categories → Products
One-to-Many
A category can have multiple products.

Customers → Orders
One-to-Many
A customer can place multiple orders.

Products → Orders
One-to-Many
A product can appear in multiple orders.

Orders → Payments
One-to-One
Each order has one payment.

DESIGN BENEFITS

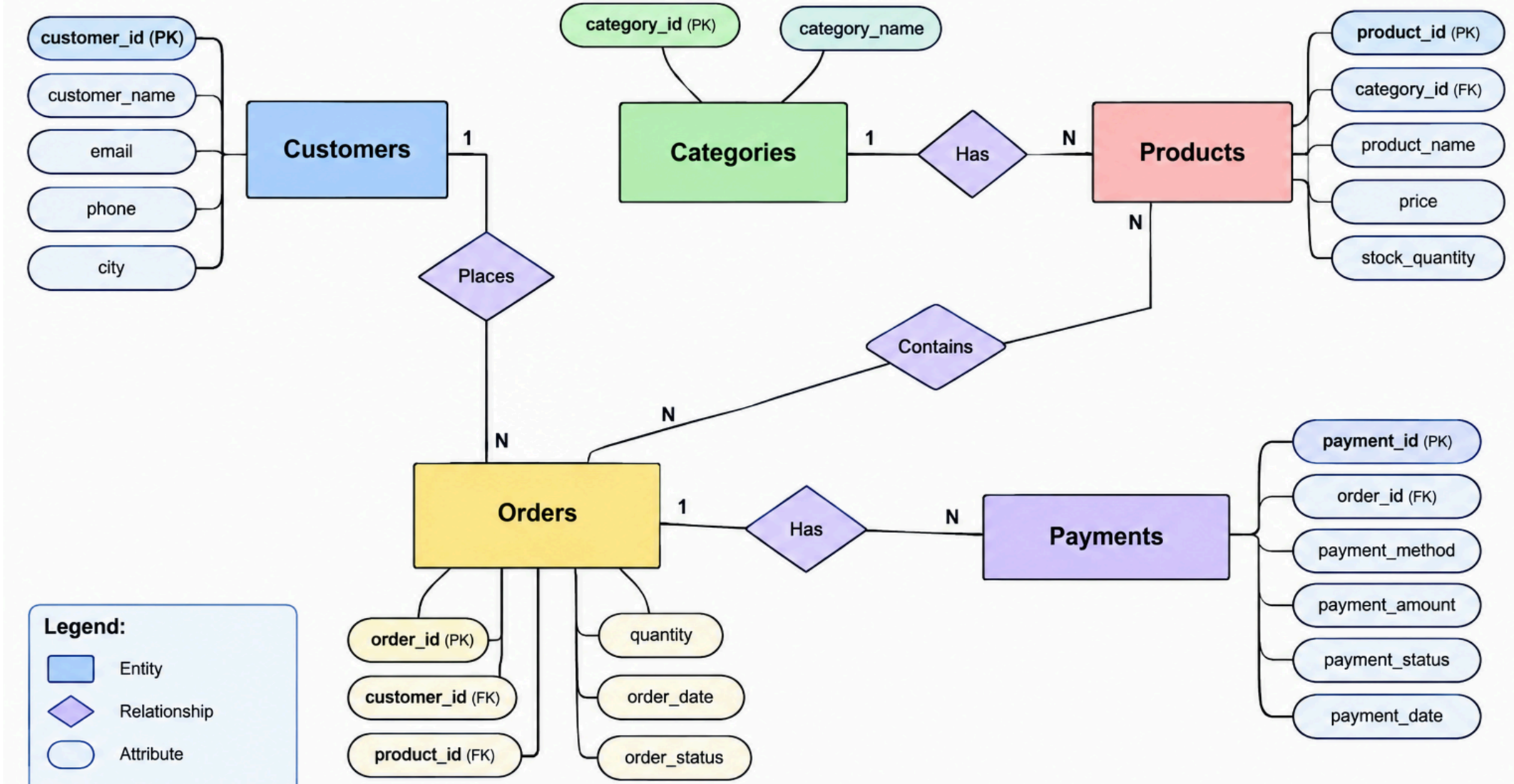
Data Consistency
Maintains accurate and reliable data.

Reduced Duplication
Avoids redundant information.

Referential Integrity
Ensures valid relationships.

Efficient Retrieval
Optimized for faster queries.

E-Commerce Database – ER Diagram



SQL Sublanguages – DDL and DML

DDL (Data Definition Language)

DDL is used to create, modify, and delete database structures.

SQL Query

```
CREATE TABLE Categories (
    category_id INT PRIMARY KEY,
    category_name VARCHAR(60)
);
```

Sample Output

STATEMENT	OUTPUT
CREATE TABLE	Query OK, 0 rows affected

DML (Data Manipulation Language)

DML is used to insert, update, and delete records in a table.

SQL Query

```
INSERT INTO Categories
VALUES (201, 'Electronics');

SELECT * FROM Categories;
```

Sample Output

STATEMENT	OUTPUT
INSERT	Query OK, 1 row affected

CATEGORY_ID	CATEGORY_NAME
201	Electronics

SQL Sublanguages – DQL and TCL

DQL (Data Query Language)

DQL is used to retrieve data from database tables.

SQL Query

```
SELECT product_name, price
FROM Products
WHERE price > 5000;
```

Sample Output

PRODUCT_NAME	PRICE
Laptop	65000.00
Smartphone	25000.00

TCL (Transaction Control Language)

TCL commands manage transactions and control changes made to the database.

SQL Query

```
START TRANSACTION;

UPDATE Products
SET stock_quantity = stock_quantity - 1
WHERE product_id = 301;

ROLLBACK;
```

Sample Output

STATEMENT	OUTPUT
START TRANSACTION	Query OK, 0 rows affected
UPDATE	Query OK, 1 row affected
ROLLBACK	Query OK, 0 rows affected

Aggregate Functions

Analyze products, inventory and sales data

SQL Query

```
SELECT
   COUNT(*) AS total_products,
   SUM(stock_quantity) AS total_stock,
   AVG(price) AS average_price,
   MAX(price) AS highest_price,
   MIN(price) AS lowest_price
FROM Products;
```

Sample Output

Metric	Value
Total Products	10
Total Stock	458
Average Price	₹12,755
Highest Price	₹65,000
Lowest Price	₹650

Functions used: COUNT(), SUM(), AVG(), MAX(), MIN()

Business value: Supports inventory monitoring, price analysis and product reporting.

String Functions

Format and display product information

SQL Query

```
SELECT
   UPPER(product_name) AS product_name,
   LENGTH(product_name) AS name_length,
   CONCAT('₹', price) AS display_price
FROM Products
LIMIT 3;
```

Sample Output

PRODUCT_NAME	NAME_LENGTH	DISPLAY_PRICE
LAPTOP	6	₹65000.00
SMARTPHONE	10	₹25000.00
BLUETOOTH SPEAKER	17	₹3000.00

Functions used: UPPER(), LENGTH(), CONCAT()

Business value: Improves product display, text formatting and report readability.

Date & Time Functions

Track orders and summarize order activity

SQL Query

```
SELECT
   order_id,
   DATE(order_date) AS order_day,
   MONTHNAME(order_date) AS order_month,
   YEAR(order_date) AS order_year
FROM Orders
LIMIT 3;
```

Sample Output

ORDER_ID	ORDER_DAY	ORDER_MONTH	ORDER_YEAR
401	2026-01-15	January	2026
402	2026-02-10	February	2026
403	2026-03-05	March	2026

Functions used: DATE(), MONTHNAME(), YEAR()

Business value: Helps generate monthly reports and analyze order trends.

Joins

Joins combine records from two or more related tables.

Types of joins: INNER JOIN • LEFT JOIN • RIGHT JOIN • CROSS JOIN

INNER JOIN

```
SELECT c.customer_name, o.order_id, o.order_date
FROM Customers c
INNER JOIN Orders o
    ON c.customer_id = o.customer_id;
```

Sample Output

CUSTOMER_NAME	ORDER_ID	ORDER_DATE
Arjun Rao	401	2026-01-15
Priya Sharma	402	2026-02-10
Kiran Reddy	403	2026-03-05

LEFT JOIN

```
SELECT p.product_name, c.category_name
FROM Products p
LEFT JOIN Categories c
    ON p.category_id = c.category_id;
```

Sample Output

PRODUCT_NAME	CATEGORY_NAME
Laptop	Electronics
Smartphone	Electronics
Bluetooth Speaker	Electronics
Phone Case	NULL

Explanation: INNER JOIN returns matching records. LEFT JOIN returns all records from the left table, along with matching records from the right table.

Subqueries

Find products priced above the average

SQL Query

```
SELECT product_name, price
FROM Products
WHERE price > (
   SELECT AVG(price)
   FROM Products
);
```

Sample Output

PRODUCT_NAME	PRICE
Laptop	65000.00
Smartphone	25000.00

How it works

1. Inner query: calculates the average product price (₹12,755).

2. Outer query: returns products priced above that average.

Subquery used: SELECT AVG(price) FROM Products

Business value: Identifies premium products for pricing analysis and business decisions.

Views

A view is a virtual table created from a SELECT query. It helps simplify frequently used queries and presents selected information.

Create the view

```
CREATE VIEW Customer_Order_Details AS
SELECT c.customer_name, p.product_name, o.quantity,
       o.order_date, pay.payment_status
FROM Customers c
JOIN Orders o ON c.customer_id = o.customer_id
JOIN Products p ON o.product_id = p.product_id
LEFT JOIN Payments pay ON o.order_id = pay.order_id;
```

To retrieve the view

```
SELECT *
FROM Customer_Order_Details;
```

Sample Output

CUSTOMER_NAME	PRODUCT_NAME	QUANTITY	ORDER_DATE	PAYMENT_STATUS
Arjun Rao	Laptop	1	2026-01-15	Paid
Priya Sharma	Smartphone	2	2026-02-10	Paid
Kiran Reddy	Bluetooth Speaker	1	2026-03-05	Pending

Stored Procedures

A stored procedure is a reusable set of SQL statements stored in the database.

Find products with low stock

```
DELIMITER //

CREATE PROCEDURE GetLowStock(
    IN stock_limit INT
)
BEGIN
    SELECT product_name,
           price,
           stock_quantity
    FROM Products
    WHERE stock_quantity < stock_limit;
END//
DELIMITER ;
```

To execute

```
CALL GetLowStock(25);
```

Sample Output

PRODUCT_NAME	PRICE	STOCK
Laptop	65000.00	12
Smartphone	25000.00	18
Bluetooth Speaker	3000.00	20

Triggers

A trigger is a database object that automatically executes when a specified event occurs on a table.

Create the trigger

```
DELIMITER //

CREATE TRIGGER prevent_negative_stock
BEFORE UPDATE ON Products
FOR EACH ROW
BEGIN
    IF NEW.stock_quantity < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Stock cannot be negative';
    END IF;
END//

DELIMITER ;
```

Test the trigger

```
UPDATE Products
SET stock_quantity = -5
WHERE product_id = 301;
```

Sample Output

```
Update rejected:
Stock cannot be negative

ERROR 1644 (45000):
Stock cannot be negative
```

CONCLUSION

- THE E-COMMERCE DATABASE MANAGEMENT SYSTEM DEMONSTRATES HOW MYSQL CAN MANAGE CUSTOMER, PRODUCT, ORDER, AND PAYMENT INFORMATION EFFICIENTLY.

- THE PROJECT USES RELATIONAL DATABASE DESIGN, SQL COMMANDS, FILTERING FUNCTIONS, JOINS, VIEWS, TRIGGERS, AND STORED PROCEDURES.

- THESE FEATURES HELP RETRIEVE INFORMATION, GENERATE REPORTS, MAINTAIN DATA CONSISTENCY, AND AUTOMATE ROUTINE DATABASE OPERATIONS.

THANK YOU